



**PSGR
Krishnammal College for Women**



SMART TRAFFIC MANAGEMENT SYSTEM
PROJECT WORK SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENT FOR THE AWARD OF THE DEGREE OF
BACHELOR OF INFORMATION TECHNOLOGY

SUBMITTED
BY
HALIMATHUS SILMIYA S (22BIT028)

GUIDED
BY
Dr. R. Jeevitha MCA., M. Phil., Ph.D.,
Assistant Professor,
Department of Information Technology,
PSGR Krishnammal College for Women,
Coimbatore-641004

MARCH-2025

CERTIFICATE

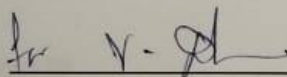
This is to certify that this project work entitled “**SMART TRAFFIC MANAGEMENT SYSTEM**” submitted to PSGR Krishnammal College for Women, Coimbatore, in partial fulfillment of the requirement for the award of the Bachelor of Information Technology, is a record of the original work done by **HALIMATHUS SILMIYA S (22BIT028)** during her period of the study in Department of Information Technology, PSGR Krishnammal College for Women, Coimbatore under my supervision and guidance and her project work has not formed the basis for the award of any Degree/Diploma/Associate/Fellowship or similar title to any candidate of any University.

Forwarded by



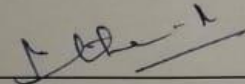
Dr. R. JEEVITHA MCA., M.Phil., Ph.D.,

Assistant Professor

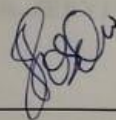


Dr. G. SOPHIA REENA MCA., M.Phil., Ph.D.,

Head of the Department



INTERNAL EXAMINER



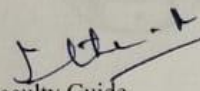
EXTERNAL EXAMINER

DECLARATION

I hereby declare that the project work entitled "SMART TRAFFIC MANAGEMENT SYSTEM" submitted to Bharathiar University, Coimbatore for the award of the Degree of Bachelor of Information Technology, is a record of original work done by **HALIMATHUS SILMIYA S (22BIT028)** under the supervision and guidance of **Dr. R. Jeevitha MCA., M. Phil., Ph.D.**, Assistant Professor, Department of Information Technology, PSGR Krishnammal College for Women, Coimbatore and that this project work has not formed the basis for the award of any degree/Diploma/Associate Fellowship or similar title to any candidate of any University.

S. Halimathus Silmiya
HALIMATHUS SILMIYA S
22BIT028

Endorsed by


Faculty Guide

Place : Coimbatore

Date : 12.03.2025

Dr. R. JEEVITHA MCA., M. Phil., Ph.D.,



07/03/2025
Coimbatore

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Ms. HALIMATHUS SILMIYA S(22BIT028)** III Year BSc Information Technology from PSGR Krishnammal College for Women, Coimbatore, has completed her Project Entitled **SMART TRAFFIC MANAGEMENT SYSTEM** in our concern from (02/12/2024 to 07/03/2025).

Her performance and conduct were found to be very good.

We wish her all the best for her future endeavours.

For TwirlTact Technology Solutions Private Limited



TABLE OF THE CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
1.	INTRODUCTION	1
	1.1 MODULE DESCRIPTION	2
	1.2 ORGANIZATION PROFILE	3
2.	SYSTEM SPECIFICATION	4
	2.1 HARDWARE SPECIFICATION	4
	2.2 SOFTWARE SPECIFICATION	11
3.	SYSTEM STUDY	13
	3.1 EXISTING SYSTEM	13
	3.2 PROPOSED SYSTEM	14
4.	SYSTEM DESIGN	16
	4.1 BLOCK DIAGRAM	16
	4.2 INPUT DESIGN	17
	4.3 OUTPUT DESIGN	21
	4.4 CODING IMPLEMENTATION	25
5.	CONCLUSION	47
	FUTURE ENHANCEMENT	48
	BIBLIOGRAPHY	49

ACKNOWLEDGEMENT

I take this opportunity with great pleasure, deep satisfaction and gratitude, the contribute of many individuals in the successful completion of the system study.

I express my hearty gratitude to **DR. R. NANDINI, Chairperson**, PSGR Krishnammal College for Women and Coimbatore for having giving us the opportunity to undertake this project.

I express my hearty gratitude to **Dr. N. YESODHA DEVI., M.Com., Ph.D., Secretary**, PSGR Krishnammal College for Women, and Coimbatore for having given us the opportunity to undertake this project.

I express my gratitude to **Dr. P. B. HARATHI., MCA, M.Phil., Ph.D., Principal**, PSGR Krishnammal College for Women, Coimbatore for her support and all resource provided.

I am extremely grateful to **Dr. G. SOPHIA REENA., MCA., M.Phil., Ph.D., Head and Associate Professor, Department of Information Technology**, PSGR Krishnammal College for Women, Coimbatore for her sustained interest and advices for the completion of this project.

I also express my heartfelt thanks to **Dr. R. JEEVITHA., MCA., M.Phil., Ph.D., Assistant Professor, Department of Information Technology**, PSGR Krishnammal College for Women, Coimbatore as a guide for her kind and patient guidance for completing this project.

I also express my special thanks **M. SANUB B.E., TWIRLTACT TECHNOLOGY SOLUTION PRIVATE LIMITED**, Coimbatore for providing us an opportunity to undertake this project in their esteemed organization continuous support to complete the project successful.

ABSTRACT

The "IoT-based Smart Traffic Management System" leverages the power of Internet of Things (IoT) devices to enhance and streamline traffic management in urban environments. The system consists of various IoT sensors, smart traffic lights, and a centralized control unit. These sensors, placed at strategic locations like intersections, roads, and pedestrian crossings, collect real-time data on vehicle density, traffic flow, and environmental conditions such as weather or roadblock information. The data is continuously transmitted to a central cloud-based platform, where it is processed and analysed.

Traffic police officers and traffic management authorities can access this data through a user-friendly interface on their mobile devices or computers. Using this information, officers can make real-time decisions to optimize traffic flow, such as adjusting traffic light timings to reduce congestion, prevent accidents, or address traffic bottlenecks caused by construction or road closures. In cases of accidents or special events that require the rerouting of traffic, officers can manually override the system to prioritize certain routes, clear traffic more effectively, or allocate resources to affected areas.

Additionally, the system is designed to handle emergency situations by providing an option for manual intervention, ensuring that emergency vehicles like ambulances, fire trucks, and police vehicles can pass through intersections quickly. The traffic lights can be programmed to automatically prioritize the passage of these vehicles, enabling them to reach their destinations faster during critical times.

The IoT-based smart traffic management system also integrates with other city infrastructure and services to provide a holistic approach to urban mobility. By using predictive analytics, the system can forecast traffic trends based on historical data and current conditions, allowing for better planning and proactive measures to prevent future congestion or accidents. Overall, this system aims to reduce traffic congestion, improve road safety, optimize the movement of goods and people, and create a more efficient and responsive urban transport network.

1 INTRODUCTION

Traffic Management is one of the biggest hurdles faced by the developing places. The growing challenges of the urban area is the traffic congestion leading to time issues. Developed countries and smart cities have already using IOT for this traffic. Even though Traditional traffic methods rely on some predefined Scheduling, pre-programmed time management, this will often lead to congestion, delays in time and when it is any Emergencies it is critical to maintain the traffic collision. To overcome this Traffic officers can monitor traffic patterns and make well-informed and pre using the data that is sent to a central control system.

The rapid urbanization and growing population have led to an increase in the number of vehicles on the road, resulting in congested traffic, decreased road safety, and increased travel times. Traditional traffic management systems rely heavily on manual intervention, which can be inefficient and ineffective in managing real-time traffic flow. Moreover, these systems often lack real-time data and insights, making it challenging for traffic managers to make informed decisions. The need for a smart and intelligent traffic management system that can optimize traffic flow, reduce congestion, and minimize environmental impact has become imperative.

This project proposes an IoT-based smart traffic management system that leverages IoT devices, such as sensors and connected traffic lights, to monitor and manage traffic flow in real-time. The system aims to provide a scalable, efficient, and sustainable solution for traffic management, enabling cities to become smarter, safer, and more environmentally friendly.

The System may prioritize emergency vehicles, control traffic during peak hours, or based on the density of the vehicle and react quickly to accidents or unusual traffic conditions by enabling traffic officers to manually intervene decisions about modifying signal timings when necessary. This data-driven, real-time strategy promises to enhance traffic flow, less congestion, and raise road safety standards. By giving traffic officer, the ability to dynamically manage traffic lights in real-time in response to crises, unique situations, or traffic conditions, this technology solves these problems. In order to gather information on vehicle volume, speed, and road conditions, IoT devices such as sensors, buzzers, and smart traffic signals are positioned strategically at intersections. Smart Traffic Management System using the Internet of Things (IoT) is an innovative solution that enhances traffic efficiency through automation and data-driven decision-making.

1.1 MODULE DESCRIPTION

The Smart Traffic Management System is a technology-driven solution that helps in efficiently managing traffic in urban areas. It utilizes various technologies like sensors, cameras, and IoT devices to monitor and manage traffics in real-time. Here's a breakdown of the module description for a smart traffic management system:

ARDUINO NANO BOARD

The Arduino Nano is a small, compact microcontroller board based on the ATmega328P microchip. It operates at 5V and has 14 digital input/output pins, 8 analog inputs, a USB connection for programming, and a power jack. It uses Arduino Nano USB cable to connect the Arduino Nano board to a computer or other USB-enabled devices for programming and communication.

NODEMCU ESP8266 BOARD

The NodeMCU is a low-cost open-source IoT development board based on the ESP8266 WiFi module. It allows you to connect to the internet easily and supports programming with Arduino IDE. It uses NodeMCU ESP8266 USB cable to connect the NodeMCU board to a computer for programming, data transfer, and power supply.

LED TRAFFIC LIGHT SIGNAL MODULE

This module consists of three LEDs (red, yellow, and green), arranged to simulate a traffic light signal. The module comes with three LEDs controlled using GPIO pins, and the timing and sequencing of the lights can be programmed through a microcontroller.

MULTIOUTPUT POWER SUPPLY

A multi-output power supply provides multiple voltage outputs (often 3.3V, 5V, and 12V) from a single unit. It typically has several regulated output channels, each designed to power different components of a project.

BREADBOARD

A breadboard is a tool for prototyping electronic circuits without soldering. It consists of a grid of holes into which components can be inserted. These holes are connected in rows and columns to allow the quick and easy construction of circuits.

LCD DISPLAY

An LCD (Liquid Crystal Display) is a screen module used to display alphanumeric characters and graphics. LCD uses the 16x2 format, meaning it has 2 rows and 16 columns of characters.

BUZZER MODULE

A buzzer module is a small sound-emitting component that can produce different tones or continuous sounds when activated. It operates with a DC voltage, and when connected to a microcontroller (like Arduino or ESP8266), it can be triggered by a digital signal.

1.2 ORGANIZATION PROFILE

Twirl Tact Technology Solutions Private Limited is Mentored by Easy Design Systems Private Limited who is an Forerunner in the software industry for more than 20+ years who have already placed in more than 11+countries across the globe & we have trained and created more than 15000+ professionals ,industry experts ,young budding engineers, academicians for various educational institutions and Board of Studies Industry Panel Expert at Various Universities like Bharathiar University, Avinashi Lingam University etc as well we are an Technology Based training platform for Schools, Colleges and universities across Southern India with expertise knowledge experience and exposure for bridging gap between industries and institutions through technical expertise by organizing Hands on Training Workshops, Faculty Development programs ,Industry Based Product Developments. Work Integrated Learning Programs, Technical Sessions and Finally placement assistance for the students who undergo programs under Twirl Tact Tech Solutions Private Limited.

End to End Technology Based hands on trainings offered for UG/PG/Scholars of Various Educational Institutions across southern India such as Bharathiar University, Coimbatore Institute of Technology, Avinashilingam University,KGiSL Group of Institutions, Hindustan Educational Institutions, PSGR Krishnammal College for Women and with over 40+ Colleges across south India on Internet of Things and Automation for Offering Valued Added Programs, Credit Courses ,Workshops, Industrial Visits, Industry Based Product Developments, Work Integrated Learning Programs and Full time Internships ,it engages students to bridge a gap between Industries and Institutions on real time working environment by developing prototypes, building products, publishing journals, patent filings and so on.

2 SYSTEM SPECIFICATION

2.1 HARDWARE SPECIFICATION

- Microcontroller - Arduino Nano Board
- IoT Module - Node MCU ESP8266 Board
- Display Module - I2c LCD Display
- Traffic Light Module - LED Traffic Lights
- Connectivity Component - Bread Board, Jumper wires
- Power Supply - Multi Output Power Supply Module, DC power adapter
- Buzzer Module

2.2 SOFTWARE SPECIFICATION

- Microcontroller Programming - Arduino IDE
- IoT Communication - Node MCU Firmware
- Alert System - Implements Buzzer Activation
- Cloud Connectivity

HARDWARE FEATURES

ARDUINO NANO BOARD

The Arduino Nano plays a crucial role in an IoT-based smart traffic management system by enabling real-time traffic monitoring, adaptive signal control, and efficient road management. Equipped with sensors like ultrasonic, IR, and RFID, the Arduino Nano detects vehicle density, pedestrian movement, and emergency vehicles at intersections. Based on this data, it dynamically controls traffic lights, buzzers, and LED displays to optimize traffic flow. Additionally, the system connects to the cloud via Wi-Fi (ESP8266) or GSM modules, allowing remote monitoring and real-time updates. In case of emergencies, such as an ambulance or fire truck approaching, the Arduino Nano can prioritize green signals to ensure faster response

times. It can also integrate with mobile apps or smart city dashboards to provide live traffic updates to drivers and authorities. The use of Arduino Nano in IoT-based traffic management enhances urban mobility, reduces congestion, and improves road safety, making transportation more efficient and intelligent.



FIG 2.1.1 Arduino NANO Board

NODEMCU ESP8266 BOARD

The Node MCU, a Wi-Fi-enabled microcontroller based on the ESP8266, plays a vital role in an IoT-based smart traffic management system by enabling real-time monitoring, adaptive traffic control, and remote access. Using sensors like IR, ultrasonic, and RFID, the Node MCU collects data on vehicle density, pedestrian movement, and emergency vehicles at intersections. It processes this data and dynamically controls traffic lights, buzzers, and LCD displays to optimize traffic flow. Since Node MCU has built-in Wi-Fi, it seamlessly connects to the cloud, allowing traffic authorities to monitor and adjust signals remotely. The system can also prioritize emergency vehicles by detecting RFID or GPS signals and turning the traffic light green. Additionally, live traffic updates can be displayed on mobile apps, websites, or digital boards to help drivers choose better routes. With its low cost, energy efficiency, and IoT capabilities, Node MCU enhances smart city traffic management by reducing congestion, improving safety, and ensuring smoother transportation.

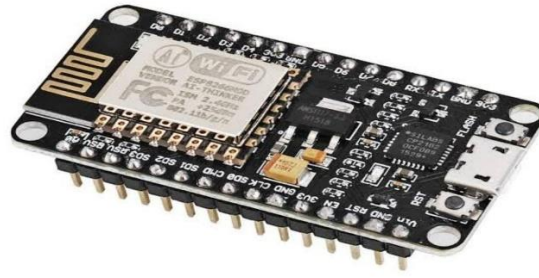


FIG 2.1.2 Node MCU Board

LCD DISPLAY

The LCD display plays a vital role in an IoT-based smart traffic management system by providing real-time visual information to drivers, pedestrians, and traffic authorities. Integrated with microcontrollers like Arduino Nano or Node MCU, the LCD receives traffic data from sensors such as IR, ultrasonic, RFID, and cameras to display essential updates. It can show traffic signal countdown timers, real-time congestion alerts, and alternative routes to reduce bottlenecks. In case of an emergency vehicle detection, such as an ambulance or fire truck, the LCD can notify other drivers to clear the way. Additionally, it can display accident alerts, road closures, weather conditions, and parking availability, improving road safety and navigation. Since the system is connected to the cloud via Wi-Fi or GSM, authorities can remotely update messages on the display for real-time traffic management. By integrating LCD displays into IoT-based traffic systems, cities can enhance traffic flow, reduce congestion, and improve public awareness, leading to a smarter and safer transportation network.



FIG 2.1.3 LCD Display

LED TRAFFIC LIGHT SIGNAL MODULE

LED traffic modules are electronic components used in traffic light systems, educational projects, and automation applications. These modules typically consist of three LEDs—red, yellow (amber), and green—arranged in a sequence to simulate real-world traffic signals. They operate using 5V or 3.3V power and can be controlled using microcontrollers like Arduino, Raspberry Pi, or ESP8266. The LEDs are driven through current-limiting resistors to ensure safe operation. These modules are commonly used in traffic management simulations, pedestrian crossing systems, and smart city projects. With simple digital outputs, they can be programmed to follow different timing sequences, mimicking real traffic light operations. Some advanced versions include additional features like built-in controllers, relays, or wireless connectivity, making them suitable for IoT-based traffic control systems.



FIG 2.1.4 Traffic Light Module

BREADBOARD

The breadboard is an essential component in an IoT-based smart traffic management system, serving as a prototyping platform for connecting various electronic components without soldering. It allows microcontrollers like Arduino Nano or Node MCU to interface with sensors (IR, ultrasonic, RFID, cameras), buzzers, LEDs, and LCD displays to create an efficient traffic management setup. Using a breadboard, components such as traffic light modules, emergency vehicle detection circuits, and pedestrian crossing systems can be interconnected for testing and fine-tuning before final implementation. It helps in developing adaptive traffic light control, where real-time data from sensors can be processed to adjust signal durations based on

vehicle density. Since IoT connectivity is crucial, the breadboard enables smooth integration of Wi-Fi (ESP8266) or GSM modules for cloud-based remote monitoring and control. By using a breadboard in smart traffic management system development, engineers can quickly test, modify, and optimize circuits, making urban transportation more efficient and intelligent.

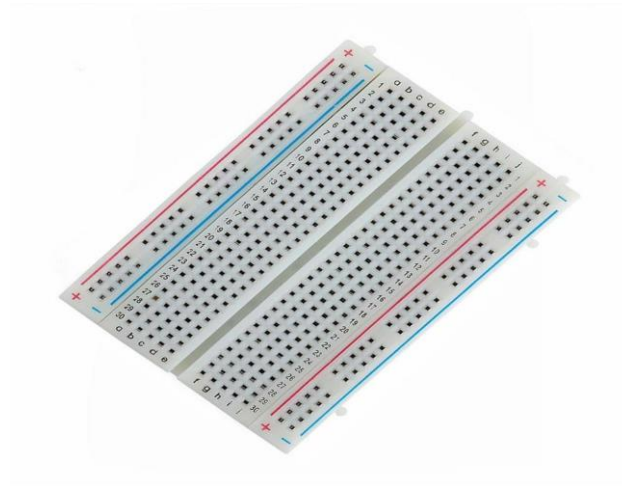


FIG 2.1.5 Bread Board

JUMPER WIRES

Jumper wires are insulated electrical wires used to connect components on a breadboard, PCB, or microcontroller without the need for soldering. They come in different types, including male-to-male, male-to-female, and female-to-female, making them compatible with various connectors and headers. Typically made of copper or aluminum, jumper wires have plastic insulation to prevent short circuits. They are available in different lengths and colors, which help in organizing and identifying connections in a circuit. Commonly used in prototyping, testing, and DIY electronics projects, jumper wires provide a quick and reusable way to create temporary connections between components.



FIG 2.1.6 Jumper Wires

MULTIOUTPUT POWER SUPPLY MODULE

The multi-output power supply module is a crucial component in an IoT-based smart traffic management system, ensuring stable and efficient power distribution to various interconnected devices. In a smart traffic system, multiple components such as Arduino Nano, NodeMCU, sensors (IR, ultrasonic, RFID), LCD displays, buzzers, LED traffic lights, and communication modules (Wi-Fi, GSM) require different voltage levels to function properly. A multi-output power supply module provides regulated voltages (e.g., 3.3V, 5V, 12V) to power these components simultaneously, ensuring reliability and efficiency. This module helps prevent voltage fluctuations and power failures, which are critical in traffic management where continuous operation is essential. Additionally, it supports solar-powered and battery backup systems, making the setup more sustainable and resilient against power outages. By integrating a multi-output power supply module, smart traffic systems can achieve stable performance, reduced energy waste, and enhanced operational reliability, contributing to a more efficient and safer transportation network.



FIG 2.1.7 Multi Output Power Supply

DC POWER ADAPTER

The DC power adapter is an essential component in an IoT-based smart traffic management system, providing a stable and reliable power source for various electronic components. Traffic management systems rely on devices such as Arduino Nano, Node MCU, sensors (IR, ultrasonic, RFID), LCD displays, LED traffic lights, buzzers, and communication modules (Wi-Fi, GSM), all of which require a consistent DC voltage supply to function properly. A DC power adapter converts AC mains power into a regulated DC output (e.g., 5V, 9V, or 12V), ensuring uninterrupted operation of the traffic system. This is especially important for critical functions like real-time traffic monitoring, adaptive signal control, and emergency vehicle detection, where power stability is crucial. Additionally, solar-powered DC adapters can be integrated to enhance sustainability and reduce dependency on the electrical grid. By using a DC power adapter, smart traffic systems can maintain efficient performance, prevent power failures, and ensure smooth traffic flow, contributing to safer and smarter urban transportation.



FIG 2.1.8 DC Power Adapter

BUZZER

The buzzer module plays a crucial role in an IoT-based smart traffic management system by providing real-time audio alerts for various traffic situations. Integrated with microcontrollers like Arduino Nano or Node MCU, the buzzer is activated based on data received from sensors such as IR, ultrasonic, and RFID. It can be used to alert pedestrians and drivers when a traffic signal changes, ensuring better compliance with traffic rules. In case of an emergency vehicle detection, such as an ambulance or fire truck, the buzzer sounds to notify other vehicles to clear the way. It can also serve as a warning system for traffic violations, such as red light jumping

or over speeding, by working alongside cameras and IoT sensors. Additionally, in accident-prone areas or hazardous weather conditions, the buzzer can provide timely alerts to prevent collisions. The system is connected to the cloud via Wi-Fi or GSM, enabling remote control and monitoring by traffic authorities. By integrating the buzzer module into smart traffic systems, cities can enhance road safety, improve traffic law enforcement, and reduce accident risks, making urban transportation more efficient and secure.



FIG 2.1.9 Buzzer

SOFTWARE FEATURES

ARDUINO IDE

The Arduino IDE (Integrated Development Environment) plays a crucial role in programming and managing the IoT-based smart traffic management system. It is used to write, compile, and upload code to microcontrollers like Arduino Nano, Node MCU, and ESP32, which control various components such as sensors (IR, ultrasonic, RFID), LED traffic lights, LCD displays, buzzers, and communication modules (Wi-Fi, GSM). With the Arduino IDE, developers can implement real-time traffic monitoring, adaptive signal control, and emergency vehicle prioritization by processing data from sensors and making dynamic adjustments. The IDE supports multiple libraries for IoT connectivity, cloud integration, and sensor interfacing, allowing the system to send traffic data to a remote dashboard for monitoring and control. By using Arduino IDE, engineers can efficiently develop, debug, and optimize smart traffic solutions, improving urban traffic flow, reducing congestion, and enhancing road safety in smart city environments.

NODEMCU FIRMWARE

The Node MCU firmware plays a vital role in IoT communication within a smart traffic management system, enabling real-time data exchange between sensors, controllers, and cloud-based platforms. Node MCU, powered by the ESP8266 Wi-Fi module, runs on Lua or Arduino-based firmware, allowing it to handle network communication efficiently. It facilitates wireless data transmission between traffic sensors (IR, ultrasonic, RFID, cameras) and a central server, enabling adaptive control of traffic lights, pedestrian signals, and emergency vehicle prioritization. The firmware allows integration with MQTT, HTTP, or WebSocket protocols, ensuring seamless connectivity between IoT devices and cloud services for remote monitoring and control. With its built-in Wi-Fi capabilities, Node MCU eliminates the need for external communication modules, making the system cost-effective and efficient. By leveraging Node MCU firmware for IoT communication, smart traffic management systems can analyse real-time data, optimize traffic flow, reduce congestion, and improve urban mobility, contributing to a smarter and safer transportation network.

CLOUD CONNECTIVITY

Cloud connectivity is a crucial component of an IoT-based smart traffic management system, enabling real-time data collection, processing, and remote monitoring. Using microcontrollers like Node MCU, ESP32, or Arduino with Wi-Fi or GSM modules, traffic data from sensors (IR, ultrasonic, RFID, cameras) is transmitted to a cloud platform such as AWS IoT, Google Cloud, or Thing Speak. This allows authorities to monitor traffic conditions, congestion levels, and emergency vehicle movement from a centralized dashboard. Cloud connectivity enables adaptive traffic signal control, where AI-driven algorithms analyse real-time data and adjust traffic lights dynamically. Additionally, drivers and pedestrians can receive live traffic updates through mobile apps, digital signboards, or SMS alerts. The cloud also supports data logging and predictive analytics, helping cities optimize traffic flow over time. By integrating cloud connectivity, smart traffic management systems become more efficient, scalable, and data-driven, reducing congestion, improving emergency response times, and enhancing overall urban mobility.

3 SYSTEM STUDY

3.1 EXISTING SYSTEM

The current traffic management system in most urban areas relies on a combination of manual control, fixed-timer traffic signals, and limited sensor-based automation. While some cities have adopted technology-driven solutions, many still operate with outdated traffic control mechanisms that do not adapt to real-time traffic conditions. The inefficiencies of the existing system lead to traffic congestion, delays, increased fuel consumption, and difficulties in emergency response.

Components of the Existing System

➤ Fixed-Timer Traffic Lights

- Traffic signals operate on pre-set timing cycles that do not change based on real-time traffic conditions.
- Leads to unnecessary waiting times at intersections, even when roads are empty.
- Causes traffic congestion during peak hours and inefficiencies in off-peak hours.

➤ Limited Use of Sensors

- Some cities have implemented induction loop sensors, which detect the presence of vehicles at intersections.
- These sensors are buried under the road and control signals based on vehicle presence.
- However, they do not provide real-time traffic density analysis and cannot predict congestion patterns.

➤ Manual Traffic Control by Police Officers

- In areas with heavy congestion or special events, traffic police manually control traffic flow.
- Officers rely on CCTV cameras or their physical presence to assess traffic conditions.
- Manual control is slow and inefficient, especially in large urban areas.

➤ No Automatic Emergency Vehicle Priority

- Emergency vehicles (ambulances, fire trucks, police) do not receive automatic priority at intersections.
- Traffic police must manually intervene to clear roads, causing delays.

- Lack of an emergency response system increases the risk of loss of life or property damage.
- Lack of Real-Time Data Collection & Cloud Integration
 - The current system does not store traffic data in a centralized cloud database.
 - Authorities cannot analyse historical traffic trends for future planning.
 - There is no predictive analytics to anticipate congestion or accidents.
- Minimal Public Traffic Alerts & Route Optimization
 - Drivers rely on basic traffic signals and road signs for navigation.
 - No real-time traffic updates are sent to mobile applications or navigation systems.
 - Commuters cannot make informed decisions to avoid congested routes.

The existing traffic management system relies heavily on fixed-timer traffic lights, limited sensors, and manual intervention, making it inefficient in handling modern urban traffic challenges. The absence of real-time data collection, emergency vehicle prioritization, and predictive analytics results in traffic congestion, longer travel times, and poor emergency response.

To overcome these challenges, the proposed IoT-based smart traffic management system will integrate smart sensors, cloud-based monitoring, AI-driven optimization, and emergency response automation to create a more efficient, safe, and responsive urban transport system.

3.2 PROPOSED SYSTEM

The proposed IoT-based smart traffic management system aims to overcome the limitations of the existing system by integrating real-time data collection, cloud computing, AI-driven optimization, and emergency vehicle prioritization. By leveraging IoT sensors, smart traffic lights, and a centralized control system, this solution ensures efficient traffic flow, reduced congestion, and improved road safety.

Components of the Proposed System

- IoT Sensors for Real-Time Traffic Monitoring
 - Infrared (IR) and Ultrasonic Sensors: Detect vehicle density at intersections.
 - CCTV & Image Processing: AI-based cameras analyse real-time traffic congestion and road conditions.

- RFID & GPS Sensors: Identify emergency vehicles and provide priority passage.
- Weather & Environmental Sensors: Detect fog, rain, or roadblocks for traffic adjustment.
- Dynamic & Adaptive Traffic Signal Control
 - Traffic light durations are dynamically adjusted based on real-time traffic density instead of using fixed timers.
 - AI-driven algorithms predict traffic conditions and optimize signal switching to minimize delays.
- Cloud-Based Data Processing & Centralized Control System
 - IoT devices send data to a cloud platform (AWS IoT, Google Cloud, Thing Speak, etc.) for processing and analysis.
 - Traffic authorities access the system remotely via a mobile app or web dashboard to monitor and control signals.
- Automated Emergency Vehicle Priority System
 - RFID/GPS sensors detect emergency vehicles and automatically switch traffic signals to green in their path.
 - Reduces delays for ambulances, fire trucks, and police cars, ensuring faster response times.
- Predictive Analytics for Traffic Forecasting
 - AI-powered analytics predict congestion trends based on historical and real-time data.
 - Authorities can take proactive measures to prevent congestion and road accidents.
- Public Traffic Alerts & Smart City Integration
 - Mobile apps, navigation systems (Google Maps, Waze), and digital signboards provide real-time traffic updates to drivers.
 - Can be integrated with public transportation and parking management systems to optimize urban mobility.

The proposed IoT-based smart traffic management system revolutionizes urban traffic control by making it real-time, data-driven, and adaptive. By integrating IoT sensors, cloud computing, AI analytics, and emergency priority systems, the solution significantly reduces traffic congestion, improves road safety, and enhances urban mobility. This system is a key step toward smart city development, ensuring a faster, safer, and more efficient transportation network.

4 SYSTEM DESIGN

4.1 BLOCK DIAGRAM

This block diagram represents an system using an Arduino Nano and a NodeMCU ESP8266 module. Here's a breakdown of the components and their functions:

- **Power Supply:** Provides the necessary power to the Arduino Nano and other connected components.
- **Arduino Nano:** Acts as the central processing unit, controlling the traffic light setup and the buzzer. It receives data from the cloud via the NodeMCU ESP8266 module.
- **Traffic Light Setup:** A set of LEDs (red, yellow, and green) that simulate a traffic light system.
- **Buzzer:** Used to give alerts, possibly for emergency vehicles, pedestrian crossings, or system notifications.
- **LCD Display:** Shows real-time status updates related to traffic lights, alerts, or messages received from the cloud.
- **NodeMCU ESP8266 Module:** Connects the system to the internet, allowing communication with a cloud-based platform.
- **Cloud:** Stores and processes traffic data, enabling remote monitoring and control of the system.
- **Mobile/Laptop:** A user interface for monitoring and controlling the system remotely via a web or mobile application.

Working:

- The Arduino Nano manages the traffic lights and buzzer based on predefined conditions or commands received from the cloud.
- The NodeMCU ESP8266 module connects to the cloud, enabling remote control and monitoring.
- Users can access the system via a mobile phone or laptop to check the traffic status or override signals if necessary.
- The LCD display provides real-time information about traffic signals and alerts.

- This system smart traffic management is useful for emergency vehicle prioritization, or adaptive signal control based on real-time data.

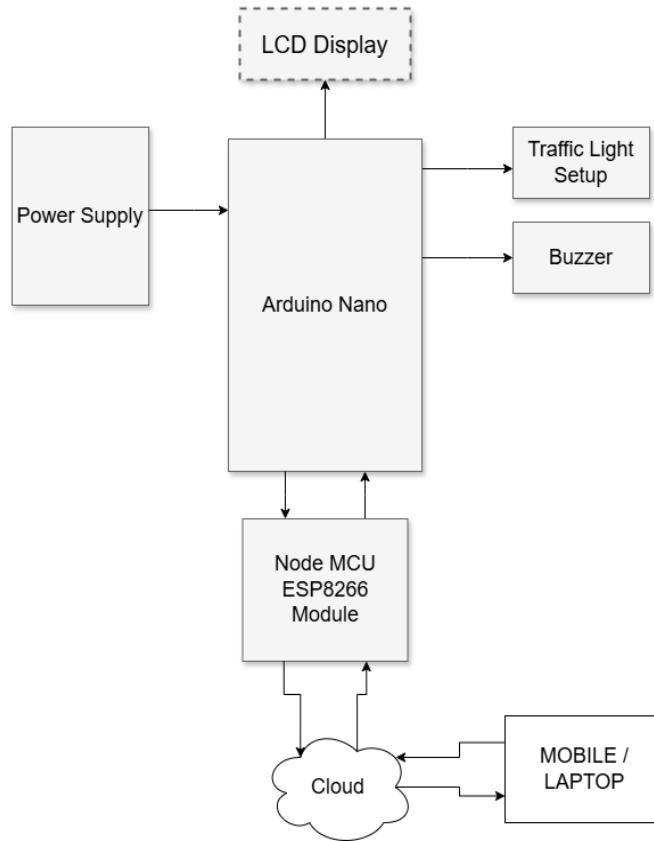


FIG 4.1.1 Block Diagram

4.2 INPUT DESIGN

The input design defines how data is collected, processed, and utilized to control the traffic management system effectively. The system integrates various hardware and software inputs to detect traffic conditions, manage emergency vehicles, and optimize traffic signals dynamically.

A. Input Components

➤ Hardware Inputs

- **Traffic Signal Control Inputs:** Connected to Arduino Nano and includes traffic light signals for four lanes. Each lane has Red, Yellow, and Green LEDs to indicate stop, caution, and go.

- Emergency Vehicle Detection: ESP8266 Module: Receives data from emergency vehicle sensors or RFID-based identification.
- Buzzer: Activates when an emergency vehicle is detected to notify traffic personnel.
- LCD Display Input: LiquidCrystal_I2C Module (0x27, 16x2) displays system status messages. Displays messages such as "IOT Based Smart Traffic," "Be Safe! Don't Drunk and Drive," and emergency alerts.
- Power Supply Input: Provides the necessary power to the Arduino Nano and connected components.
- Software Inputs
- ESP8266 Communication Inputs: Data is received from the ESP8266 module via Software Serial (A0, A1).

The system checks for emergency vehicle alerts:

"lane1_on" → Emergency detected in Lane 1

"lane2_on" → Emergency detected in Lane 2

"lane3_on" → Emergency detected in Lane 3

"lane4_on" → Emergency detected in Lane 4

Data is processed to change traffic lights dynamically.

- Cloud-Based Inputs (Optional for Future Enhancements): Data sent to/from the cloud allows remote monitoring via a mobile/laptop interface. Can integrate real-time traffic analysis and manual control options.

B. Input Processing Mechanism

- Sensor Data Reception: The ESP8266 module communicates with Arduino Nano to detect emergency vehicles. Data is received via the Software Serial interface.
- Traffic Light Control Logic: The system follows a sequence of operations (op1 to op8) to manage signal changes. If an emergency vehicle is detected, the current lane's green light is prioritized, and other lanes turn red.
- Alert System Activation: If emergency data is received, the buzzer activates. The LCD display shows emergency messages like "Lane1 Emergency Vehicle!".

- Manual Override Possibilities (Future Scope): Cloud-based inputs can allow traffic authorities to control lights remotely. Future enhancement could include a mobile app for manual intervention.

C. Importance of Input Design

- Efficient Traffic Flow: Adjusts signal timings based on real-time traffic and emergency detection.
- Enhanced Road Safety: Provides priority for emergency vehicles, reducing response time.
- Automation & Smart Decision Making: Uses microcontroller-based logic to dynamically adjust traffic lights.
- Scalability: Can be integrated with additional sensors or cloud-based remote control features in the future.

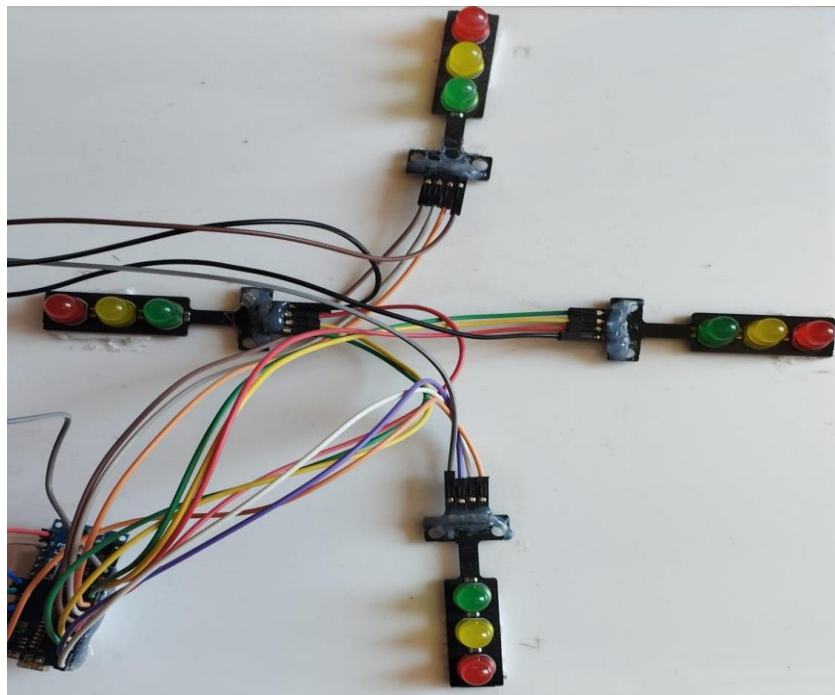


FIG 4.2.1 Input Traffic Light



FIG 4.2.2 Input LCD Display

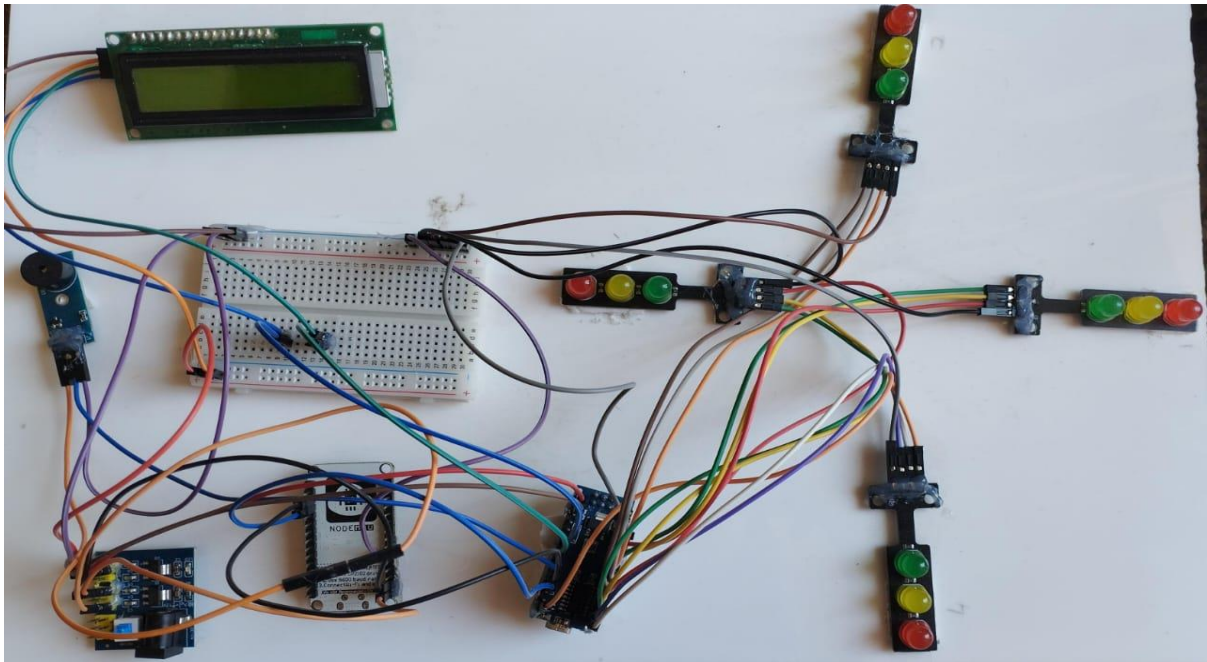


FIG 4.2.3 Input Design

Flowchart Description

- Start
- Receive Input Data

Power On System

Sensor Inputs (Traffic Light Sensors, Emergency Vehicle Detection)

Cloud Inputs (Mobile/Laptop Commands)

- Process Data in Arduino Nano

Normal Traffic Control

Emergency Vehicle Detection

- Check for Emergency Vehicle

Yes: Override normal traffic, prioritize the emergency lane, activate buzzer & update LCD.

No: Continue normal traffic sequence.

- Send Data to Cloud (For remote monitoring & control)
- Repeat Process

4.3 OUTPUT DESIGN

The output design ensures that the system responds effectively to real-time traffic conditions, emergency vehicle detection, and user interactions. It defines how data is presented to users and how traffic signals, alerts, and displays are controlled based on system inputs.

A. Output Components

➤ Hardware Outputs

- Traffic Signal Control (LEDs): Each lane has Red, Yellow, and Green LEDs. The system dynamically switches these lights based on normal traffic flow and emergency detection. Priority control for emergency vehicles by making their lane green.
- Emergency Alert System (Buzzer): Activates when an emergency vehicle is detected. Alerts surrounding traffic to give way. Stops buzzing when emergency clearance is completed.
- LCD Display (16x2 I2C): Displays real-time system status messages, such as:

Startup Messages:

"IOT Based Smart Traffic"

"Management System"

Traffic Safety Messages:

"Be Safe! Don't Drunk and Drive"

Emergency Vehicle Alerts:

"Lane1 Emergency Vehicle!"

"Lane2 Emergency Vehicle!"

"Lane3 Emergency Vehicle!"

"Lane4 Emergency Vehicle!"

➤ Software and Cloud-Based Outputs

- Serial Monitor (Arduino Serial Output): Displays received data from the ESP8266 module. Prints lane activation messages, emergency vehicle detection, and system debug information.
- Cloud-Based Output (ESP8266 & Mobile/Laptop Interface): The ESP8266 sends traffic data to the cloud. Allows remote monitoring via a mobile/laptop interface. Future enhancements could include real-time traffic updates and manual control options.

B. Output Processing Mechanism

- Traffic Light Control Sequence: The system executes an eight-step traffic light sequence (op1 to op8). When an emergency vehicle is detected, the respective lane turns green, overriding the normal sequence.
- Emergency Vehicle Detection and Alert Mechanism: The ESP8266 module detects emergency vehicle presence ("laneX_on"). The buzzer activates, and the LCD display updates with an emergency message. Once the vehicle passes, normal operation resumes.
- Cloud Integration for Remote Monitoring: Traffic data can be stored and visualized on a cloud platform. Future updates could allow manual intervention via a mobile app.

C. Importance of Output Design

- Ensures clear communication with road users (traffic lights, LCD messages).
- Provides priority to emergency vehicles while maintaining normal traffic flow.
- Enhances road safety by alerting drivers to potential hazards.
- Supports future scalability with cloud-based remote control and monitoring.

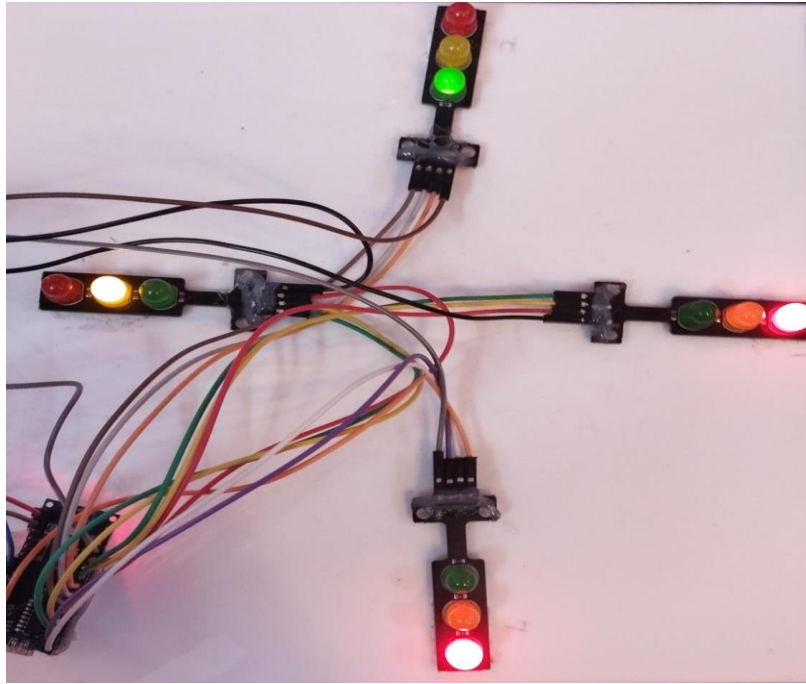


FIG 4.3.1 Output Traffic Light



FIG 4.3.2 Output LCD Display



FIG 4.3.3 Output LCD Display

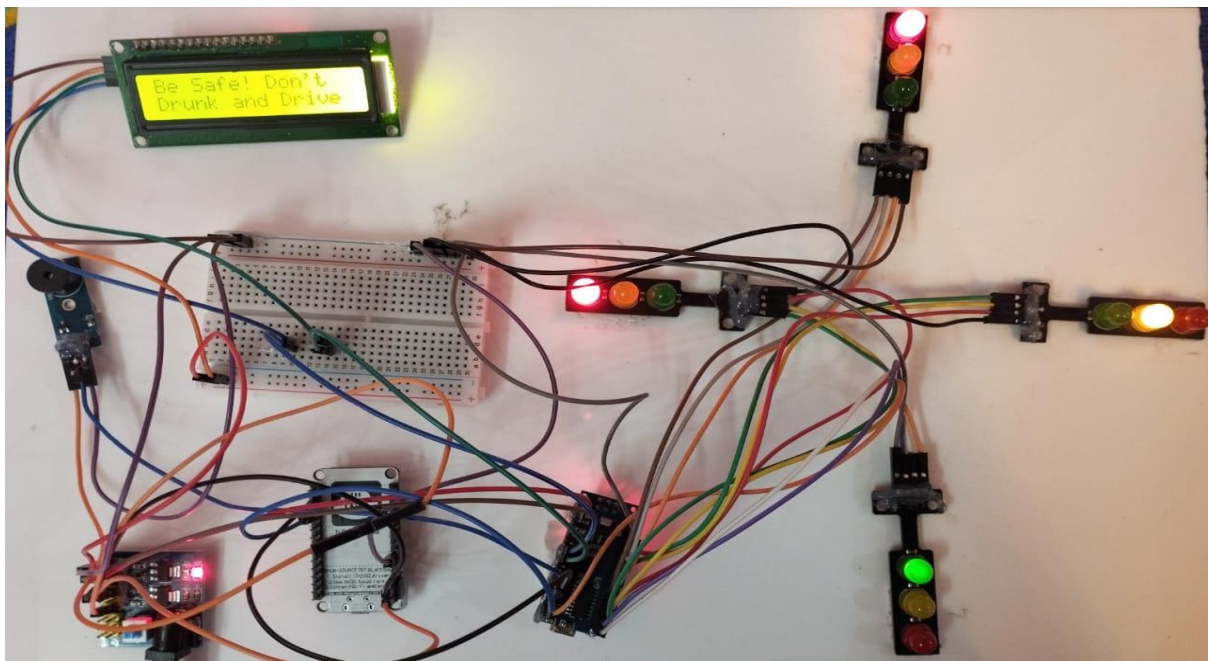


FIG 4.3.4 Output Design

Flowchart Description

- System Initialization

LCD displays startup messages.

All traffic lights are set to default states (Red/Green/Yellow as per sequence).

- Normal Traffic Light Sequence Execution

The system runs op1 to op8, cycling through lanes.

LCD displays safety messages.

- Emergency Vehicle Detection (ESP8266 Communication)

If an emergency vehicle is detected in a lane, the respective “laneX_on” signal is received.

Normal sequence is overridden.

Traffic light for the detected lane turns GREEN, others turn RED.

Buzzer activates, LCD displays “LaneX Emergency Vehicle!”.

- Post-Emergency Handling

Once the emergency vehicle passes, the buzzer stops.

The system resumes normal traffic light sequence.

- Cloud-Based Monitoring

Data is sent to the cloud via ESP8266.

Mobile/Laptop interface displays real-time traffic status.

4.4 CODING IMPLEMENTATION

ARDUINO NANO CODE

```
#include <SoftwareSerial.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define buzzerpin A2
```

```
int Lane1[] = {2,3,4}; // Lane 1 Red, Yellow and Green
```

```
int Lane2[] = {5,6,7}; // Lane 2 Red, Yellow and Green
```

```
int Lane3[] = {8,9,10}; // Lane 3 Red, Yellow and Green
```

```
int Lane4[] = {11,12,13}; // Lane 4 Red, Yellow and Green
```



```

int count=0;

int flag = 0;

String data_esp1="";

SoftwareSerial arduinoSerial(A0,A1);

LiquidCrystal_I2C lcd(0x27,16,2);

void setup()

{

  Serial.begin(9600);

  arduinoSerial.begin(9600);

  pinMode(buzzerpin,OUTPUT);

  digitalWrite(buzzerpin,HIGH);

  lcd.init();

  lcd.clear();

  lcd.backlight();    // Make sure backlight is on

  // Print a message on both lines of the LCD.

  lcd.setCursor(0,0);

  lcd.print("IOT Based Smart");

  lcd.setCursor(4,1);

  lcd.print("Traffic");

  delay(2000);

  lcd.clear();

  // Print a message on both lines of the LCD.

  lcd.setCursor(3,0);

  lcd.print("Management");

```

```

lcd.setCursor(5,1);

lcd.print("System");

delay(2000);

lcd.clear();

for (int i = 0; i < 3; i++)

{

    pinMode(Lane1[i], OUTPUT);

    pinMode(Lane2[i], OUTPUT);

    pinMode(Lane3[i], OUTPUT);

    pinMode(Lane4[i], OUTPUT);

}

for (int i = 0; i < 3; i++)

{

    digitalWrite(Lane1[i], LOW);

    digitalWrite(Lane2[i], LOW);

    digitalWrite(Lane3[i], LOW);

    digitalWrite(Lane4[i], LOW);

}

}

void loop()

{

    op1();

    op2();

    op3();

```

```

    op4();

    op5();

    op6();

    op7();

    op8();

}

void lcd_print()

{

    lcd.clear();

    lcd.setCursor(0,0);

    lcd.print("Be Safe! Don't");

    lcd.setCursor(0,1);

    lcd.print("Drunk and Drive");

}

void op1()

{

    static int time1 = 10; // Persistent variable

    for(time1 = 10;time1>0;time1--)

    {

        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

        checkFun3();

```

```

checkFun4();

if(flag == 1)

{

    digitalWrite(Lane1[0], LOW);

    flag = 0;

}

digitalWrite(Lane4[2], LOW);

digitalWrite(Lane4[0], HIGH);

digitalWrite(Lane1[1], LOW);

digitalWrite(Lane1[2], HIGH);

digitalWrite(Lane2[0], HIGH);

digitalWrite(Lane3[0], HIGH);

delay(1000);

}

}

void op2()

{

    static int time2=5;

    for(time2 = 5;time2>0;time2--)

    {

        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

```

```

    checkFun3();

    checkFun4();

    if(flag == 1)
    {
        digitalWrite(Lane2[0], LOW);

        digitalWrite(Lane1[2], HIGH);

        flag = 0;
    }

    digitalWrite(Lane2[0], LOW);

    digitalWrite(Lane2[1], HIGH);

    delay(1000);
}

}

void op3()
{
    static int time3=10;

    for(time3=10;time3>0;time3--)

    {

        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

        checkFun3();

        checkFun4();

```

```

if(flag == 1)
{
    digitalWrite(Lane2[0], LOW);
    flag = 0;
}

digitalWrite(Lane1[2], LOW);
digitalWrite(Lane1[0], HIGH);
digitalWrite(Lane2[1], LOW);
digitalWrite(Lane2[2], HIGH);
delay(1000);
}
}

void op4()
{
    static int time4=5;
    for(time4=5;time4>0;time4--)
    {
        lcd_print();
        receive_fun();
        checkFun1();
        checkFun2();
        checkFun3();
        checkFun4();
        if(flag == 1)

```

```

{
    digitalWrite(Lane3[0], LOW);

    digitalWrite(Lane2[2], HIGH);

    flag = 0;
}

digitalWrite(Lane3[0], LOW);

digitalWrite(Lane3[1], HIGH);

delay(1000);
}
}

void op5()
{
    static int time5=10;

    for(time5=10;time5>0;time5--)
    {
        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

        checkFun3();

        checkFun4();

        if(flag == 1)
        {
            digitalWrite(Lane3[0], LOW);

```

```

    flag = 0;

}

digitalWrite(Lane2[2], LOW);

digitalWrite(Lane2[0], HIGH);

digitalWrite(Lane3[1], LOW);

digitalWrite(Lane3[2], HIGH);

delay(1000);

}

}

void op6()

{

    static int time6=5;

    for(time6=5;time6>0;time6--)

    {

        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

        checkFun3();

        checkFun4();

        if(flag == 1)

        {

            digitalWrite(Lane4[0], LOW);

            digitalWrite(Lane3[2], HIGH);

```



```

    flag = 0;

}

digitalWrite(Lane4[0], LOW);

digitalWrite(Lane4[1], HIGH);

delay(1000);

}

}

void op7()

{

    static int time7=10;

    for(time7=10;time7>0;time7--)

    {

        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

        checkFun3();

        checkFun4();

        if(flag == 1)

        {

            digitalWrite(Lane4[0], LOW);

            flag = 0;

        }

        digitalWrite(Lane3[2], LOW);

```

```

    digitalWrite(Lane3[0], HIGH);

    digitalWrite(Lane4[1], LOW);

    digitalWrite(Lane4[2], HIGH);

    delay(1000);

}

}

void op8()

{

    static int time8=5;

    for(time8=5;time8>0;time8--)

    {

        lcd_print();

        receive_fun();

        checkFun1();

        checkFun2();

        checkFun3();

        checkFun4();

        if(flag == 1)

        {

            digitalWrite(Lane1[0], LOW);

            digitalWrite(Lane4[2], HIGH);

            flag = 0;

        }

        digitalWrite(Lane1[0], LOW);

```

```

    digitalWrite(Lane1[1], HIGH);

    delay(1000);

}

}

void receive_fun()

{

    if(arduinoSerial.available()>0) // It will only send data when the received data is greater
    than 0

    {

        data_esp1 = arduinoSerial.readStringUntil('\n'); //reads the incoming serial data from the
        serial buffer in the string

        data_esp1.trim(); // removes all these leading/trailing whitespaces from the string.

        Serial.println(data_esp1);

    }

}

void checkFun1()

{

    while(data_esp1=="lane1_on")

    {

        digitalWrite(buzzerpin,LOW);

        lcd.clear();

        lcd.setCursor(0,0);

        lcd.print("Lane1 Emergency");

        lcd.setCursor(2,1);

        lcd.print("Vehicle!");

```

```

if(count==0)
{
    for (int i = 0; i < 3; i++)
    {
        digitalWrite(Lane1[i], LOW);
        digitalWrite(Lane2[i], LOW);
        digitalWrite(Lane3[i], LOW);
        digitalWrite(Lane4[i], LOW);
    }
}

digitalWrite(Lane1[2], HIGH);
digitalWrite(Lane3[0], HIGH);
digitalWrite(Lane4[0], HIGH);
digitalWrite(Lane2[0], HIGH);
receive_fun();

count=1;
}

if(count==1)
{
    digitalWrite(buzzerpin,HIGH);
    digitalWrite(Lane1[2], LOW);
    digitalWrite(Lane1[0], HIGH);
    count=0;

    flag = 1;
}

```

```

    }
}

void checkFun2()
{
    while(data_esp1=="lane2_on")
    {
        lcd.clear();

        lcd.setCursor(0,0);

        lcd.print("Lane2 Emergency");

        lcd.setCursor(0,1);

        lcd.print("Vehicle!");

        digitalWrite(buzzerpin,LOW);

        if(count==0)
        {
            for (int i = 0; i < 3; i++)
            {
                digitalWrite(Lane1[i], LOW);

                digitalWrite(Lane2[i], LOW);

                digitalWrite(Lane3[i], LOW);

                digitalWrite(Lane4[i], LOW);

            }
        }

        digitalWrite(Lane1[0], HIGH);

        digitalWrite(Lane3[0],HIGH);
    }
}

```

```

    digitalWrite(Lane4[0], HIGH);

    digitalWrite(Lane2[2], HIGH);

    receive_fun();

    count=1;

}

if(count==1)

{

    digitalWrite(buzzerpin,HIGH);

    digitalWrite(Lane2[2], LOW);

    digitalWrite(Lane2[0], HIGH);

    count=0;

    flag = 1;

}

}

void checkFun3()

{

    while(data_esp1=="lane3_on")

    {

        lcd.clear();

        lcd.setCursor(0,0);

        lcd.print("Lane3 Emergency");

        lcd.setCursor(0,1);

        lcd.print("Vehicle!");

        digitalWrite(buzzerpin,LOW);

```

```

if(count==0)
{
    for (int i = 0; i < 3; i++)
    {
        digitalWrite(Lane1[i], LOW);
        digitalWrite(Lane2[i], LOW);
        digitalWrite(Lane3[i], LOW);
        digitalWrite(Lane4[i], LOW);
    }
}

digitalWrite(Lane1[0], HIGH);
digitalWrite(Lane3[2],HIGH);
digitalWrite(Lane4[0], HIGH);
digitalWrite(Lane2[0], HIGH);
receive_fun();

count=1;
}

if(count==1)
{
    digitalWrite(buzzerpin,HIGH);
    digitalWrite(Lane3[2], LOW);
    digitalWrite(Lane3[0], HIGH);
    count=0;

    flag = 1;
}

```

```

    }
}

void checkFun4(){
    while(data_esp1=="lane4_on")
    {
        lcd.clear();

        lcd.setCursor(0,0);

        lcd.print("Lane4 Emergency");

        lcd.setCursor(0,1);

        lcd.print("Vehicle!");

        digitalWrite(buzzerpin,LOW);

        if(count==0)
        {
            for (int i = 0; i < 3; i++)
            {
                digitalWrite(Lane1[i], LOW);

                digitalWrite(Lane2[i], LOW);

                digitalWrite(Lane3[i], LOW);

                digitalWrite(Lane4[i], LOW);

            }
        }

        digitalWrite(Lane1[0], HIGH);

        digitalWrite(Lane3[0],HIGH);

        digitalWrite(Lane4[2], HIGH);
    }
}

```



```

digitalWrite(Lane2[0], HIGH);

receive_fun();

count=1;

}

if(count==1)

{

digitalWrite(buzzerpin,HIGH);

digitalWrite(Lane4[2], LOW);

digitalWrite(Lane4[0], HIGH);

count=0;

flag = 1;

}

}

```

NODEMCU CODE

```

// Nodemcu iot code json fprmat receive

#include <PubSubClient.h> //Allows a client to publish/subscribe messaging with a MQTT
server

#include <ESP8266WiFi.h> //Used to connect ESP8266 to the Wi-Fi network

#include <WiFiClient.h> //Creates a client that can connect to a specified internet IP address
and port

#include <ESP8266WebServer.h> //provides HTTP Client and Web Socket Client

#include <SoftwareSerial.h> //enables serial communication with a digital pin other than the
serial port

SoftwareSerial espSerial(D2,D3); // set up a new serial object

```

```

// #define MAX_SRV_CLIENTS 3 // Variable for maximum 3 clients only

// #define mqtt_user "" // Variable for mqtt broker username

// #define mqtt_password "" //Variable for mqtt broker password

#define WiFi_led D4

const char* ssid= "Silmiya"; // Enter your access point name

const char* password = "silmiya2004"; // Enter your access point password

const char* mqtt_server= "test.mosquitto.org";

WiFiClient wificlient; // Initialize the Wi-Fi client library

PubSubClient client(wificlient); // Initialize the PubSubClient library

// Function to connect to Wi-Fi

void connectToWiFi() {

    //WiFi.config(addr, addr, NMask); // manually set the network addresses

    WiFi.begin(ssid, password); //Start the connection with wifi network

    Serial.print("\nConnecting to : ");

    Serial.println(ssid); //print the connected wifi name

    while (WiFi.status() != WL_CONNECTED)

    {

        delay(500);

        Serial.print(".");

        digitalWrite(WiFi_led,LOW);

    }

    digitalWrite(WiFi_led,HIGH);

    Serial.println("");

    Serial.println("WiFi Connected");

```

```

Serial.println("IP address: ");

Serial.println( WiFi.localIP()); // prints the localIP

delay(25);

}

void setup()

{

  Serial.begin(9600); // Set serial baud rate to 9600

  espSerial.begin(9600); // Set software serial baud rate to 9600

  digitalWrite(WiFi_led,HIGH);

  connectToWiFi();

  client.setServer(mqtt_server, 1883); //connecting to a mqtt broker

  client.setCallback(callback); // Set the message received callback

  delay(500);

}

String str="";

void loop() { // This function runs over and over again in a continuous loop

  if (!client.connected()) // Check if we're connected to the MQTT broker

  {

    reconnect(); // If we're not, attempt to reconnect

  }

  client.loop(); //Call the loop to maintain connection to the server

}

void reconnect() {

  while (!client.connected()) // Loop until we're reconnected

```

```

{
    Serial.print("Attempting MQTT connection...");

    if (client.connect("Traffic System"))
    {
        Serial.println("connected");

        client.subscribe("Traffic/command/topic"); // Subscribe to the topic where our web page is
publishing messages

        digitalWrite(WiFi_led,HIGH);
    }
    else {
        digitalWrite(WiFi_led,LOW);

        Serial.print("failed, rc=");

        Serial.print(client.state());

        Serial.println(" try again in 5 seconds");

        delay(5000); // Wait some time to space out connection requests
    }
}

void callback(char* topic, byte* payload, unsigned int length) // This function handles
received messages
{
    Serial.print("Command from MQTT broker is : [");

    Serial.print(topic);

    Serial.print("] : ");

    String message = ""; // Convert the message payload from bytes to a string

```

```

for (unsigned int i=0; i< length; i++) {

    message = message + (char)payload[i];

}

Serial.println(message);

espSerial.print(message); // Print the message to the software serial port

Serial.println();

} //end callback

```

4.4.1 METHODOLOGY

This Arduino-based traffic light control system is designed for a four-lane intersection, where each lane has three LEDs representing the red, yellow, and green traffic signals. This consists of separate functions (lan1fun() , lane2fun(), lane3fun(), and lane4fun()) that control the lights for each lane. When a function like lane2fun() is called, Lane 2's green light turns ON, allowing vehicles to pass, while the red lights remain ON for all other lanes, ensuring they stop. Similarly, for lan1fun() , lane3fun() and lane4fun() make Lan 1, Lane 3 and Lane 4 green, respectively, while keeping all other lanes red.

The yellow lights are always turned OFF and turn on for a particular seconds while moving to green meaning there is no transition phase between green and red lights. This system operates sequentially, meaning one lane at a time gets the green signal, and the transitions can be managed by calling these functions in the loop() function with appropriate delays. However, for a more realistic and safer traffic system, yellow lights should be introduced before switching to red to alert drivers. Additionally, this system can be further enhanced by adding pedestrian crossing signals, timers, or even real-time traffic sensors to manage traffic flow dynamically based on congestion. Implementing such improvements would make the traffic control system more efficient and adaptable to real-world scenarios.

5 CONCLUSION

The IoT-based Smart Traffic Management System is designed to improve traffic efficiency by integrating sensors, microcontrollers, and wireless communication. It utilizes an Arduino-based control system to manage traffic lights at a four-way intersection, where each lane has red, yellow, and green signals. The system follows a sequential operation to regulate traffic flow, ensuring smooth and systematic movement of vehicles. Additionally, it is equipped with an emergency vehicle detection mechanism that prioritizes ambulances, fire trucks, or police vehicles. When an emergency vehicle is detected, the system overrides the normal traffic sequence, immediately turning the corresponding lane's light green while activating a buzzer and displaying alerts on an LCD screen. This feature ensures that emergency services can pass through without delay, improving response times and potentially saving lives.

Communication between the system and an external module, such as ESP8266, enables real-time traffic monitoring and remote control capabilities. The system receives data indicating emergency vehicle presence or other traffic conditions and adapts accordingly. By implementing IoT technology, the system can be further enhanced with AI-based traffic density analysis, which would allow it to dynamically adjust signal timings based on real-time congestion levels. This would reduce unnecessary waiting times and lower fuel consumption, contributing to a more sustainable urban environment.

Overall, the IoT-based Smart Traffic Management System is a practical and innovative solution for modern cities struggling with congestion and inefficient traffic control. It not only optimizes vehicle movement but also enhances road safety by accommodating emergency vehicles efficiently. Future advancements could integrate machine learning algorithms, vehicle-to-infrastructure (V2I) communication, and adaptive signal control, making traffic systems more intelligent and responsive to real-world conditions.

FUTURE ENHANCEMENT

The current IoT-based Smart Traffic Management System effectively regulates traffic flow and prioritizes emergency vehicles. However, there are several areas where future enhancements can make the system more intelligent, adaptive, and efficient. One significant improvement is AI-driven traffic optimization, where real-time traffic density detection using cameras and sensors can help dynamically adjust signal timings. Implementing computer vision and machine learning algorithms would allow the system to recognize congestion patterns and allocate green light duration accordingly, reducing unnecessary waiting times and fuel consumption.

Another crucial enhancement is Vehicle-to-Infrastructure (V2I) communication. By integrating IoT-enabled traffic lights with connected vehicles, the system can receive real-time location data from approaching vehicles, further refining signal control. Emergency vehicles, buses, and public transport can be automatically prioritized without needing additional sensors.

Cloud-based traffic monitoring and data analytics can also improve system efficiency. By storing and analysing historical traffic data, city authorities can make data-driven decisions, predict congestion trends, and optimize road infrastructure planning. Real-time traffic data can be displayed on mobile applications, providing drivers with alternative route suggestions to reduce congestion.

For smart city integration, the traffic management system can be connected with smart parking systems and public transport networks. This would help regulate traffic based on the availability of parking spaces or the schedule of buses and trains, further improving urban mobility.

Solar-powered traffic lights and low-power IoT devices can enhance the system's sustainability, reducing electricity consumption and enabling deployment in areas with limited power infrastructure. Additionally, 5G connectivity can improve communication speed between traffic signals and the control centre, making real-time updates more efficient. Future upgrades may also include voice command integration for emergency responders to manually override signals via secure channels in case of critical situations.

BIBLIOGRAPHY

1. "IoT-based Smart Traffic Management System for Smart Cities"

Authors: [Check IEEE Xplore or ResearchGate]

Summary: Discusses IoT sensors, AI, and cloud computing in managing urban traffic.

Link: Search IEEE Xplore

2. "Real-time Traffic Monitoring Using IoT and Machine Learning"

Focuses on using edge computing, smart sensors, and AI for traffic control.

Available on: Springer, ScienceDirect

3. "Smart Traffic Light Control System Using IoT"

Uses RFID, IR sensors, and AI for adaptive traffic signal control.

Available on: Google Scholar, ACM Digital Library

4. Yueyue Na, Yanmeng Guo, Qiang Fu, and Yonghong Yan, "An Acoustic Traffic Monitoring System: Design and Implementation", UIC-ATC-ScalCom-CBDCom -IoP.2015.41

5. Sk Riyazhussain, Riyazhussain, C.R.S. Lokesh, P.Vamsikrishna, Goli Rohan, "Raspberry Pi Controlled Traffic Density Monitoring System", IEEE WiSPNET 2016 conference.

6. Dinkar Sitaram, Nirupama Padmanabha, Supriya S, Shibani S, "Still

Image Processing Techniques for Intelligent Traffic Monitoring", 2015

Third International Conference on Image Information Processing.